

C++ STUDY GUIDE

sort · find · count

정렬, 검색, 개수 세기를 한 번에 이해하는 STL 알고리즘 정리

범위의 의미 · 반환값 · 실수 포인트 · 자료구조별 사용법

한 문장 요약 sort()는 데이터를 정렬하고, find()는 원하는 값의 첫 번째 위치를 찾으며, count()는 원하는 값이 몇 개인지 계산합니다.

정리 기준: 첨부된 대화 원문

작성일: 2026 년 8 월 12 일

1. 먼저 보는 핵심 비교

함수	무엇을 하나?	원본 변경	반환값	대표 용도
sort()	범위를 정렬	변경함	없음	오름차순·내림차순 정렬
find()	첫 번째 일치 값을 검색	변경 안 함	찾은 위치(반복자)	존재 여부·첫 위치 확인
count()	일치 값의 개수를 계산	변경 안 함	개수	중복 횟수 계산

필수 헤더 세 함수 모두 <algorithm>에 들어 있습니다: #include <algorithm>

2. 공통 개념: [시작, 끝) 범위

세 알고리즘은 대부분 '시작 위치부터 끝 위치 직전까지'를 처리합니다. 끝 위치는 마지막 원소가 아니라 마지막 원소의 다음 위치입니다.

```
algorithm(container.begin(), container.end(), ...);
// begin()은 첫 원소, end()는 마지막 원소 다음 위치
```

위치	의미
begin()	첫 번째 원소를 가리킴
end()	마지막 원소 다음 위치를 가리킴; 역참조하면 안 됨

기억법 범위는 [begin, end)처럼 끝을 포함하지 않습니다. 그래서 전체 범위는 begin()과 end()를 그대로 넘기면 됩니다.

3. sort() — 순서를 정렬하기

sort()는 범위 안 원소의 순서를 직접 바꾸는 제자리(in-place) 정렬입니다. 정렬된 복사본을 새로 만드는 함수가 아닙니다.

기본 형태

```
sort(시작_위치, 끝_위치);
sort(시작_위치, 끝_위치, 비교_기준);
```

간단 예제: 오름차순

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numbers = {5, 2, 8, 1, 3};
    sort(numbers.begin(), numbers.end());

    for (int number : numbers) cout << number << " ";
}
```

출력: 1 2 3 5 8

내림차순과 일반 배열

```
sort(numbers.begin(), numbers.end(), greater<int>()); // 8 5 3 2 1

int data[5] = {5, 2, 8, 1, 3};
sort(data, data + 5); // data + 5 직전까지 정렬
```

추가 헤더 greater<int>()를 쓰려면 일반적으로 #include <functional>을 함께 포함합니다.

sort() 실수 포인트

- #include <algorithm>을 빠뜨리지 않습니다.
- end() - 1 을 넘기면 마지막 원소가 정렬에서 제외됩니다.
- set 은 이미 정렬되어 있으며 임의의 위치 이동이 불가능하므로 std::sort 를 적용할 수 없습니다.

4. find() — 원하는 값의 첫 위치 찾기

find()는 범위에서 값이 처음 나타나는 위치를 반복자(iterator)로 반환합니다. 값 자체가 아니라 위치를 돌려준다는 점이 핵심입니다.

기본 형태

```
auto result = find(시작_위치, 끝_위치, 찾을_값);
```

간단 예제

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numbers = {10, 20, 30, 40, 50};
    auto result = find(numbers.begin(), numbers.end(), 30);

    if (result != numbers.end())
        cout << "찾은 값: " << *result;
    else
        cout << "값을 찾지 못했습니다.";
}
```

출력: 찾은 값: 30

안전 규칙 result 가 end()가 아닌지 확인한 뒤에만 *result 로 값을 꺼냅니다. end()는 유효한 원소가 아닙니다.

인덱스 구하기

```
if (result != numbers.end()) {
    int index = distance(numbers.begin(), result);
    cout << index << "번 인덱스"; // 30 은 2 번 인덱스
}
```

문자열 검색

```
vector<string> books = {
    "Harry Potter", "C Programming", "Data Structure"
};
```

```
auto result = find(books.begin(), books.end(), "C Programming");
```

문자열은 전체 내용과 대소문자까지 같아야 일치합니다. 예를 들어 "C"나 "c programming"은 찾지 못합니다.

find() 실수 포인트

- `cout << result;`처럼 반복자를 값처럼 출력하지 않습니다. 실제 값은 `*result` 입니다.
- 찾지 못했는데 `*result` 를 사용하면 정의되지 않은 동작이 생길 수 있습니다.
- 중복 값이 있어도 첫 번째 일치 위치만 반환합니다.

5. count() — 같은 값의 개수 세기

`count()`는 범위 전체를 확인해 특정 값과 정확히 같은 원소가 몇 개인지 반환합니다.

기본 형태와 간단 예제

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numbers = {1, 2, 2, 3, 2, 4};
    int result = count(numbers.begin(), numbers.end(), 2);
    cout << "2의 개수: " << result;
}
```

출력: 2의 개수: 3

조건으로 세려면 count_if()

```
int result = count_if(
    numbers.begin(), numbers.end(),
    [](int number) { return number > 2; }
);
```

구분 `count()`는 정확히 같은 값을 세고, `count_if()`는 람다 같은 조건을 만족하는 값을 셉니다.

count() 실수 포인트

- 반환값은 위치가 아니라 개수입니다.
- 문자열도 전체 내용과 대소문자가 정확히 같아야 셉니다.
- 존재 여부만 필요하면 첫 일치에서 멈출 수 있는 find()가 의도를 더 잘 드러냅니다.

6. 세 함수를 한 번에 비교하는 예제

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numbers = {3, 1, 2, 3, 5};

    sort(numbers.begin(), numbers.end());
    auto position = find(numbers.begin(), numbers.end(), 3);
    int amount = count(numbers.begin(), numbers.end(), 3);

    cout << "정렬 결과: ";
    for (int number : numbers) cout << number << " ";
    cout << '\n';

    if (position != numbers.end())
        cout << "첫 번째 3의 인덱스: "
              << distance(numbers.begin(), position) << '\n';

    cout << "3의 개수: " << amount << '\n';
}
```

정렬 결과: 1 2 3 3 5
 첫 번째 3의 인덱스: 2
 3의 개수: 2

실행 흐름

1. sort()가 원본을 1, 2, 3, 3, 5 순서로 바꿉니다.
2. find()가 첫 번째 3의 위치를 찾아 인덱스 2를 가리킵니다.
3. count()가 전체 범위를 검사해 3이 두 개임을 반환합니다.

7. vector, 배열, set에서의 권장 방식

자료구조	권장 검색 방식	비고
vector	find(v.begin(), v.end(), 값)	일반 STL 알고리즘 사용
일반 배열	find(arr, arr + 크기, 값)	포인터 범위를 전달
set	s.find(값)	컨테이너 전용 멤버 함수 권장

unordered_set	s.find(값)	컨테이너 전용 멤버 함수 권장
----------------------	-----------	------------------

set 은 중복 값을 저장하지 않으므로 s.count(값)의 결과는 항상 0 또는 1 입니다.

```
set<int> numbers = {10, 20, 30};
cout << numbers.count(20); // 1
cout << numbers.count(50); // 0
```

8. 시험·발표 직전 체크리스트

- 세 함수 모두 <algorithm> 헤더를 포함했는가?
- 범위의 끝은 마지막 원소가 아니라 마지막 원소 다음 위치인가?
- sort()가 원본 순서를 직접 바꾼다는 점을 설명할 수 있는가?
- find() 결과를 end()와 비교한 뒤 역참조하는가?
- count()의 반환값이 개수라는 점을 구분하는가?
- 조건 검색·조건 집계가 필요하다면 find_if()·count_if()를 떠올리는가?

최종 암기 문장 sort() = 정렬, find() = 첫 위치, count() = 개수. 세 함수는 '시작부터 끝 직전까지'의 범위를 처리합니다.